

Part 6: CompletableFuture | Future and Callable in Java

```
public static void main(String args[]) {  
  
    ThreadPoolExecutor poolExecutor = new ThreadPoolExecutor(1, 1, 1, TimeUnit.HOURS, new ArrayBlockingQueue<>(10),  
        Executors.defaultThreadFactory(), new ThreadPoolExecutor.AbortPolicy());  
  
    //new thread will be created and it will perform the task  
    poolExecutor.submit(() -> {  
        System.out.println("this is the task, which thread will execute");  
    });  
  
    //main thread will continue processing  
}
```

New thread will be created here: lets say thread1

Now, what if caller want to know the status of the **thread1**. Whether its completed or failed etc.

Vấn đề: Khi gửi một task vào trong Thread Pool thực thi, nếu tác vụ đó là một Runnable thì, ta sẽ không có cách nào biết được task đó nó đã hoàn thành hay thất bại.

1. Future

- là một interface đại diện cho kết quả của một task bất đồng bộ
- Nó cho phép ta kiểm tra:
 - Việc tính toán đã xong chưa
 - Lấy kết quả
 - Xử lý ngoại lệ nếu có

2. Callable

- đại diện cho một tác vụ cần thực thi giống như Runnable
- Nhưng:
 - Runnable không có kiểu trả về
 - Callable có kiểu trả về

Có 3 dạng phương thức submit:

- `Future<?> submit(Runnable task)` : Nhận một đối tượng `Runnable` (không trả về kết quả). Trả về một đối tượng `Future` có giá trị `null` khi tác vụ hoàn thành.
- `<T> Future<T> submit(Runnable task, T result)` : Nhận một đối tượng `Runnable` và một giá trị kết quả mặc định `result`. Trả về `Future<T>` và khi tác vụ xong, hàm `get()` sẽ trả về chính giá trị `result` đó.
- `<T> Future<T> submit(Callable<T> task)` : Nhận một đối tượng `Callable` có khả năng trả về kết quả hoặc ném ngoại lệ. Trả về `Future<T>` chứa kết quả sau khi tính toán xong

3. CompletableFuture

- được giới thiệu ở Java 8
- Future có nhược điểm đó là hàm `get()` sẽ làm block luồng hiện tại cho đến khi có kết quả. CompletableFuture giúp chạy các tác vụ ở luồng riêng mà không block luồng chính.
- Đây được xem là phiên bản nâng cao của `Future`, cung cấp thêm khả năng **chaining**

a. supplyAsync:

- Phương thức `supplyAsync` nhận vào một `Supplier` và khởi tạo một task bất đồng bộ.
- Tác vụ này sẽ được thực thi trong một luồng riêng biệt.
- Ta có thể truyền vào 1 Executor để kiểm soát luồng chặt chẽ hơn
- Nếu không truyền vào 1 Executor thì mặc định nó sẽ dùng chung Fork-Join Pool, nơi có thể tự động điều chỉnh pool size dựa vào vi xử lý của máy

b. thenApply & thenApplyAsync:

- Nhận kết quả từ task Async trước đó, và apply vào 1 function để biến đổi dữ liệu
- trả về một đối tượng `CompletableFuture`

c. `thenCompose` & `thenComposeAsync`: Dùng để móc nối các task Async phụ thuộc vào nhau (tác vụ sau cần kết quả của tác vụ trước).

d. `thenAccept` & `thenAcceptAsync`: Đây thường là giai đoạn cuối cùng (end stage) trong chuỗi. Nó nhận vào kết quả để xử lý (ví dụ: in ra màn hình, lưu xuống database) nhưng **không trả về bất cứ thứ gì** (kiểu Void)

e. `thenCombine` & `thenCombineAsync`: Dùng khi có 2 tác vụ `CompletableFuture` chạy độc lập, và muốn gom (combine) kết quả của cả 2 lại với nhau khi chúng đều đã hoàn thành.

☒ ~~tìm hiểu về `AbortPolicy` và các loại exception khác *****~~

☒ ~~dùng `future.get()` vẫn phải đợi, thread pool executor thực hiện xong thì mới thực hiện được đoạn code tiếp theo; vậy nó có gì khác với code bình thường không có đa luồng~~

☐ dùng `thenCompose` nhiều lần, sau đó gọi 3 lần `thenComposeAsync`, chuyện gì xảy ra? (liên kết với bài ForkJoin Pool)

☒ ~~lấy ví dụ về sự khác nhau của `thenCompose` và `thenApply` trong trường hợp trả về `CompletableFuture` trùng nhau~~